

Topics Under HTML Form and Tables

Forms

- HTML Form Structure and Method (POST, GET)
- Input Types: Text, Email, Number, Date, Time, Password, select>option
- Labels, IDs, name and required, readonly
- Checkbox, Radio Buttons, and Select (Single/Multi)
- Submit and **Reset Buttons** and Form Submission

Tables

- HTML Table Structure: Rows, Columns, and Cells
- Semantic Tables with thead, tbody, and tfoot
- Table Attributes: **colspan**, **rowspan**, classes, and IDs
- Special Table Styling : **<colgroup>/<col>**
- Size Chart Example: E-Commerce Style Table with thead and tbody

****Important Element and Attributes of Forms****

What is Labels?

An element that used to define a **label** for an `<input>` (or other form controls) is the `<label>` tag. It improves **accessibility** and **usability** by associating descriptive text with a form control, allowing users to click the label to focus or activate the input.

What is ID?

ID is a selector that is used to select unique element from the Document (HTML)

1. ID can be used to create Links (Eg. **#hero-section**)
2. ID can also be used to apply styling (In CSS)
3. **In Forms: Id is used to create associated text using `<label>` Element with `for=""` attribute that links input's **id** attribute**

Two ways to create labels in form input field ?

1. Explicit (We've done this)
2. Implicit

What is **name** attribute in HTML?

The **name** in an <input> element is an **attribute** that specifies a name for the input field.

- It is mainly used to **identify form data** when a form is submitted and ~~to reference the element in JavaScript.~~
- When a form is submitted, only inputs with a **name** attribute will have their values sent to the server
- The **name** becomes the key in the key-value pair sent in the request.

What is **readonly** attributes?

Note: readonly and required attributes are **boolean attributes**

- Makes the input **non-editable** by the user, but still **focusable** and **submittable** with the form.

What is **required** attributes?

- Marks the input as **mandatory** for form submission and doesn't allow to submit the form until required field is provided by client

What is **value** attributes?

- The **value** in an <input> tag specifies the **initial or current value** of the input element, and its meaning depends on the input type.

Types of Button in Forms ?

Submit Button: Sends the form data to the server by validating form fields and redirect to action Page (Default Behaviours)

Reset Button: Resets all form fields to their **initial values**

Types of Methods in Forms?

GET : Form data is appended to the URL as query parameters (e.g., ?name=John&age=25).

Data is visible in the browser's address bar.

POST: Form data is sent **in the request body**, not in the URL, data is not visible in address bar

What is **action** attributes in form?

The action attribute in an HTML <form> element specifies **where to redirect** and **send the form-data when the form is submitted**

Tables element in HTML ?

The **<table>** element is used to represent **tabular data** — information arranged in rows and columns .

A table is made up of several related elements:

Emp Code	Emp Name	Job Title	Salary(LPA)
911	Zoya Shaikh	Developer	3.6
912	Lisa Dev	Tester	2.8
913	Deepak Jadeja	Developer	5.2
914	Aiti Yewale	Digital Marketing	4
915	Simran Rao	HR	5.6

<table>	Defines the table container.
<tr>	Defines a table row (where all <td> are inserted).
<th>	Defines a header cell (<u>bold and centered by default</u>).
<td>	Defines a standard data cell (<u>Actual data is passed here</u>).
<caption>	Adds a title or description for the table.
<thead>	Groups header rows. (All <th> are wrapped under this)
<tbody>	Groups body rows.
<tfoot>	Groups footer rows.

-----STYLING COLUMNS-----

<colgroup> / <col> The <colgroup> tag in HTML is used to group columns within a table and apply styles to them collectively.

Example:

```
<colgroup>
  <col span="" style="background-color: lightblue" />
  <col style="background-color: lightgreen" />
  <col style="background-color: red" />
</colgroup>
```

Special Attributes in Table

colspan: Specifies how many columns a cell **<td>** should span.

```
<td colspan="2">Content</td>
```

rowspan : Specifies how many rows a cell **<td>** should span.

```
<td rowspan="3">Content</td>
```

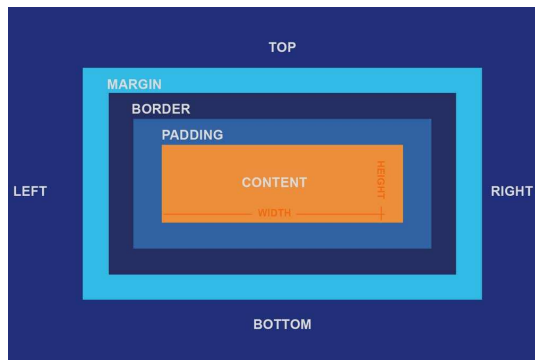
Let Understand what is BOX MODEL

Every Element in the HTML have **these four Properties** that can be modified based on the need!

Components of the Box Model

1. **Content**: This is the innermost part of the box where the **actual content**, such as **text** or **images**, is displayed.
2. **Padding**: This is the space between the content and the border. It is transparent and can be used to create space inside the element.
3. **Border**: This surrounds the padding and content. It can be styled with different colors, widths, and patterns.
4. **Margin**: This is the outermost layer that creates space between the element and other elements on the page. It is also transparent.

Demonstration with Example:



Apply styling to Table

```
td{  
  border: 2px solid rgb(247, 241, 241);  
  padding: 10px;  
}  
table{  
  border-collapse: collapse;  
}
```

What is Git and Github



Git:

Git is a **version control Software** system that **records** and **manages** changes to **files over time**, enabling developers to **track history**, **revert to earlier versions**, and collaborate without overwriting each other's work.

Created by Linus Torvalds in 2005, and is now the industry standard for managing source code efficiently.

Key Features of Git (Why do we need)?

1. **Version Tracking:** Every change is recorded with **author**, **timestamp**, and **commit message**
2. **Efficient Branching & Merging:** Create isolated branches for features or fixes, then merge them into the main codebase
3. **Collaboration:** Multiple developers can work in parallel without conflicts.
4. **Distributed Architecture:** Each clone is a complete backup of the project.
5. **Commit History:** A clear log of all changes for auditing and debugging

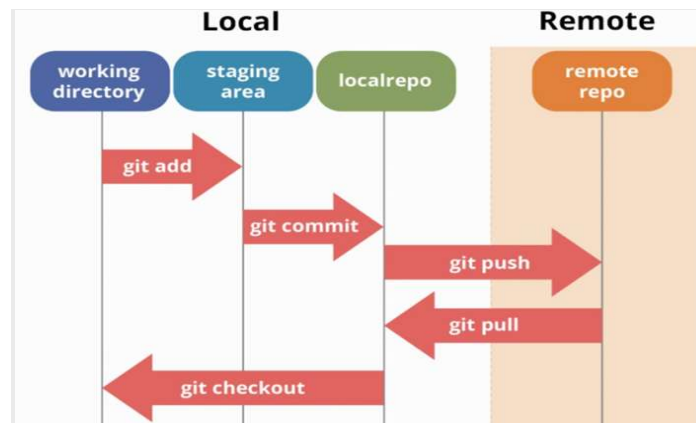
Git as a Distributed Version Control System

Git is a **Distributed Version Control System (DVCS)**, meaning every developer has a **complete local copy** of the repository, including its full history. This is different from **Centralized Version Control Systems (CVCS)(Github)** where there is a single central repository and developers must connect to it for most operations.

Git Concepts

- **Repository-** A directory where **git** monitors your project files and records their revision history.
- **Clone-** It created a **local copy of a remote repository** on your machine.
- **Stage -** It selects specific changes that you want Git to include in the next snapshot.
- **Commit-** It records the staged **changes as a permanent version in the project history**.
- **Branch-** Lets you **develop new features or experiment without affecting the main project**.
- **Merge-** Integrates changes from one branch into another.
- **--** It fetches and applies updates from a remote repository to your local one.
- **Push-** It Uploads your local commits to a remote repository

Git Repository Structure
It consists of 4 parts:



1. **Working directory:** This is your local directory where you make the project (write code) and make changes to it. (Cmd: `git init`)
2. **Staging Area (or index):** this is an area where you first need to put your project before committing. This is used for code review by other team members. (Cmd : `git add.`)
3. **Local Repository:** this is your local repository where you commit changes to the project before pushing them to the central repository on GitHub. This is what is provided by the distributed version control system. This corresponds to the `.git` folder in our directory. (Cmd: `git commit -m "message"`)
4. **Central Repository:** This is the main project on the central server, a copy of which is with every team member as a local repository. (Cmd: `git push -u origin main`)

Github:

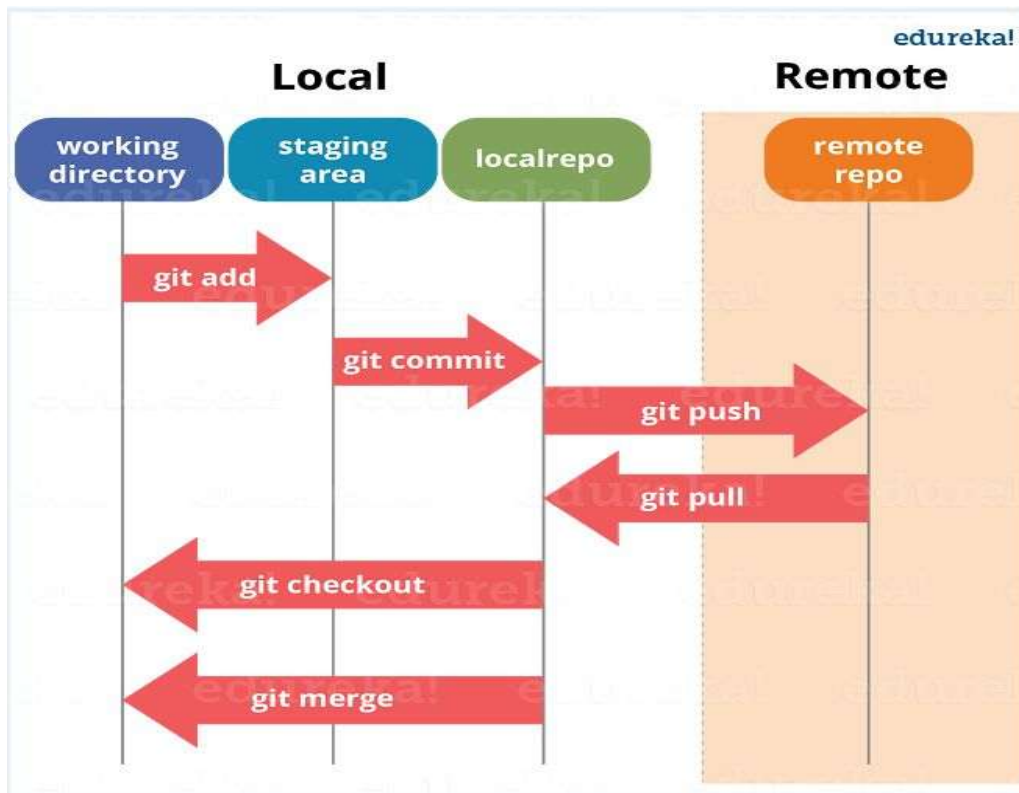
Github is a web-based hosting service and cloud platform owned by Microsoft. It acts as a remote warehouse where developers backup and share local Git Repositories online.



- Installing Git and Signup in Github
- Setting user profile with email and username

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```


- Workflow of Git



- **Initialize Repositories :** This command will “create local Repository”

```
laxma@Laksh MINGW64 /d/Trainx (main)
• $ git init
Reinitialized existing Git repository in D:/Trainx/.git/
```

This creates a new hidden folder called **.git**

- **Git add Code to Staging Area:**

```
laxma@Laksh MINGW64 /d/Trainx (main)  
$ git add .
```

- Git commit to add code in **local repositories**

```
laxma@Laksh MINGW64 /d/Trainx (main)  
$ git commit -m "added practise sett"
```

- The “**git clean**” command is a powerful tool in **Git that helps you remove untracked files and directories** from your working directory.

Git Branch Commands

The git branch command is used to **list**, **create**, **rename**, and **delete** branches in a Git repository. **Branches allow you to work on different features or fixes independently without affecting the main codebase.**

- **To create Branch and Switch:**
`git branch feature-login` **OR** `git checkout -b branch_name` (To create and Checkout)
- **Listing Branches**
`git branch`
- **To switch to another Branch**
`git checkout branch_name`
- **Rename Branch**
`git branch -M old-branch-name new-branch-name`
- **Delete Branch**
`git branch -d <branch-name>` # Delete if merged
`git branch -D <branch-name>` # Force delete even if unmerged

How to Merge a Git Branch into Master?

WHAT: Merging branches in Git allows you to combine code changes from different lines of development.

WHY -> This is Important for:

- **Feature Integration:** Bringing new features into the main codebase.
- **Bug Fixes:** Applying bug fixes to the main branch quickly.
- **Code Consistency:** Keeping your codebase consistent and free of conflicts

HOW TO Merge

- First switch to `main` branch then RUN -- >
“ `git merge branch_name_to_merge` (into main)

Difference Between git push and git pull

Git Push: **git push** is used to upload local commits to a remote repository. It transfers changes from your **local branch** to the **corresponding branch on the remote repository**.

Key Features:

- Updates the remote branch to match your local branch.
- Sends your local commits to the remote repository.
- Requires the remote branch to be in sync with your local branch (no conflicting changes).
- Commonly used after committing changes locally.

Git Pull: **git pull** is used to **fetch** and **merge** changes from a remote repository into your local branch. It combines two operations.

Key Features:

- Retrieves updates from the remote branch and integrates them into your local branch.
- May result in merge conflicts if both local and remote branches have conflicting changes.

Difference Between git fetch and git clone

Git clone: **git clone** is used when you want to create a full local copy of a remote repository for the first time. **It downloads all files, commit history, branches, and tags, and sets up a remote connection (usually named origin).**

Note: After cloning, you have a complete working directory and .git folder containing the repository's history.

Git Command:

Clone a remote repository to your local machine

```
>> git clone https://github.com/user/repo.git
```

Git Fetch: **git fetch** is used after you **have already cloned a repository**. It contacts the remote repository and **downloads any new commits, branches, or tags that you don't yet have locally, but it does not automatically merge them into your working branch.** This allows you to review changes before integrating them.

Git Command:

Fetch updates from the remote repository

```
git fetch origin
```

To Logs all the Commits

Logs Commits: `git log` (To list down all the commits with author, email, date and commit message, commit hash)

Logs Commits in One Line: `git log --oneline` (To list down all the commits hash and message)

Difference BTW 2 commits

Command: `git logs commitHash commitHash`

What is Tag and why do we need?

Let understand what the flow of Software development is

Development → Testing → Release Version → Deployment

* Tag

A lightweight tag in Git is essentially a simple pointer to a specific commit, without storing extra metadata like the tagger's name, date, or message. It's useful for quick, internal markers rather than public releases.

Create Tag: `git tag <tagname>` (This will automatically attached last commit)

Create Tag with Hash: `git tag <tagname> <commitHash>`

List Tags: `git tag`

Details about Tags: `git show <tagname>`

Delete Tags: `git tag -d <tagname>`

Push Single Tags: `git push origin <tagname>`

Push all tags: `git push origin -tags`

What is the use of Tag?

To release the versions of the software (That show what you've implemented in that version like fixes, feature)

TO RELEASE VERSION - GITHUB

1. Step to Tags in Github
2. Release Version
3. Select Tag to Release
4. You're DONE